CoRT: Code-integrated Reasoning within Thinking

Taro Yamada[1]

[1]Anonymous Institution

Abstract

Large Reasoning Models (LRMs) show impressive chain-of-thought capabilities but struggle with precise mathematical computations, leading to brittleness in mathematical problem-solving. While Code Interpreters (CIs) can provide deterministic computation, models often fail to strategically integrate code within their reasoning process. We present CoRT (Code-integrated Reasoning within Thinking), a novel approach that teaches LRMs to strategically integrate code execution within their thinking process through targeted hint-engineering and structured post-training. Our method introduces two key innovations: (1) minimal but effective hints that guide models to trigger code usage at appropriate moments and trust CI outputs, and (2) a three-stage training pipeline combining supervised fine-tuning (SFT), rejection fine-tuning (RFT), and code-aware reinforcement learning (RL). CoRT achieves substantial improvements over strong baselines across five mathematical benchmarks (AIME24/25, AMC23, MATH500, OlympiadBench), with our best 32B model achieving 81.8% average accuracy while using 50% fewer tokens than comparable approaches. Our 1.5B model with RL training reaches 58.3% average accuracy, outperforming similar-sized baselines by +8 percentage points. CoRT demonstrates that strategic code integration through targeted prompting and lightweight post-training can unlock efficient, high-accuracy mathematical reasoning.

1 Introduction

Mathematical reasoning represents one of the most challenging frontiers for large language models (LLMs). While recent Large Reasoning Models (LRMs) like OpenAI's o1 [1] and DeepSeek-R1 [**?**] demonstrate impressive chain-of-thought capabilities, they remain brittle when precise numerical computations are required. The integration of Code Interpreters (CIs) offers a promising solution by providing deterministic computation capabilities, yet models often struggle with fundamental questions: *when* to invoke tools, *how* to incorporate outputs effectively, and *how* to maintain coherent reasoning flow.

Current approaches to tool-augmented mathematical reasoning face several critical limitations. First, models frequently fail to recognize appropriate moments for tool invocation, leading to inefficient manual calculations where code execution would be more reliable. Second, even when tools are used, models often exhibit verification-heavy behavior, unnecessarily double-checking deterministic computational results instead of trusting them. Third, the field lacks high-quality training data that demonstrates strategic code integration within reasoning processes.

We introduce CoRT (Code-integrated Reasoning within Thinking), a novel framework that addresses these challenges through two key innovations: *hint-engineering* for strategic tool use and a *three-stage post-training pipeline* for robust code-integrated reasoning. Our approach teaches models to view code execution as a natural extension of their thinking process, rather than as an external tool requiring constant verification.

Preprint.

CoRT's hint-engineering strategy uses minimal but carefully designed prompts to guide model behavior at critical moments. A simple cold-start hint after the thinking delimiter boosts code-triggering probability from 50% to 90%, while targeted hints encourage code usage during tedious calculations and discourage unnecessary re-verification of computational outputs. This approach requires only 30 high-quality exemplars to generate effective training data.

Our three-stage training pipeline combines supervised fine-tuning (SFT) on synthesized data, rejection fine-tuning (RFT) for improved response quality, and reinforcement learning (RL) with code-aware rewards. The RL stage incorporates both accuracy rewards and code execution rewards ($R = R_a + \omega R_c, \omega = 0.1$) to encourage appropriate tool use patterns.

Experimental results demonstrate substantial improvements across five challenging mathematical benchmarks. Our best 32B model (Prompt-Hint-SFT-32B) achieves 81.8% average accuracy, matching QwQ-32B's performance with approximately 50% fewer tokens. For lightweight models, our Prompt-Hint-1.5B-RL reaches 58.3% average accuracy, outperforming the best baseline by +8 percentage points while maintaining computational efficiency.

Our key contributions are:

- A hint-engineered data synthesis framework that overcomes training data scarcity with minimal high-quality exemplars
- An efficient post-training pipeline (SFT $\rightarrow$ RFT $\rightarrow$ RL) with code-integrated rollouts, persistent environments, and code execution rewards
- Strong empirical results with substantial token efficiency improvements across five mathematical benchmarks
- Behavioral analysis revealing a shift from verification-heavy to balanced calculation-verification code usage patterns

2 Related Work

Mathematical Reasoning with LLMs Mathematical reasoning has emerged as a key benchmark for evaluating LLM capabilities. The MATH dataset [2] established a challenging testbed for mathematical problem-solving, while recent work has introduced more specialized benchmarks including AIME [2] and olympiad-style problems [2]. Early approaches relied primarily on chain-of-thought prompting [3], but recent advances have incorporated reinforcement learning and process supervision to improve reasoning quality.

Tool-Augmented Reasoning The integration of external tools, particularly code interpreters, has shown promise for enhancing mathematical reasoning capabilities [3]. However, most existing approaches treat tool use as an auxiliary capability rather than an integral part of the reasoning process. Recent work on tool-learning [3] has explored training models to use tools more effectively, but these methods often require extensive tool-specific training data.

Large Reasoning Models The emergence of o1-style models [1] has demonstrated the power of extended reasoning during inference. DeepSeek-R1 [**?**] and QwQ-32B [4] have shown that these capabilities can be achieved through reinforcement learning on reasoning tasks. However, these models still struggle with precise numerical computations and often exhibit inefficient token usage patterns.

Reinforcement Learning for Code Generation Recent advances in applying RL to code generation [**?**] have shown promising results. Work on code-augmented reasoning [3] has explored combining natural language reasoning with code execution, but most approaches focus on standalone coding tasks rather than integrated mathematical reasoning.

Our work differs from existing approaches by focusing on strategic integration of code execution within the reasoning process itself, using minimal hints to guide behavior and a structured training pipeline to achieve robust code-integrated reasoning.

3 Methodology

3.1 Problem Formulation

We formulate code-integrated mathematical reasoning as an iterative process where models alternate between natural language reasoning and code execution within a persistent computational environment. Given a mathematical problem p, the model generates a reasoning trace $\tau = \{t_1, t_2, \ldots, t_n\}$ where each t_i can be either natural language text or executable code. The goal is to produce a final answer a that correctly solves the problem while efficiently utilizing computational resources.

3.2 Hint-Engineering Framework

CoRT introduces two complementary approaches for guiding strategic code integration: *Prompt-Hint* and *Hint-Engineering*.

Prompt-Hint Approach The Prompt-Hint method uses a simple but effective cold-start prompt immediately after the `<think>` delimiter: *"Okay, let's try to solve this problem step by step using multiple python code calls"* This single intervention dramatically increases code-triggering probability from approximately 50% to 90%, encouraging models to consider computational approaches early in their reasoning process.

Hint-Engineering Approach Building on the cold-start prompt, Hint-Engineering incorporates additional targeted cues throughout the reasoning process:

- **Calculation triggers**: When manual computation appears tedious, hints like *"It looks tedious, and we can use python code"* encourage tool usage
- **Trust mechanisms**: Hints such as *"We don't need to doubt the accuracy of python calculations"* discourage unnecessary re-verification of deterministic outputs
- **Strategic guidance**: Context-specific prompts guide appropriate moments for code invocation

Figure 1 illustrates both approaches using Problem 13 from AIME23 I, showing how hints guide the reasoning process toward more efficient code-integrated solutions.

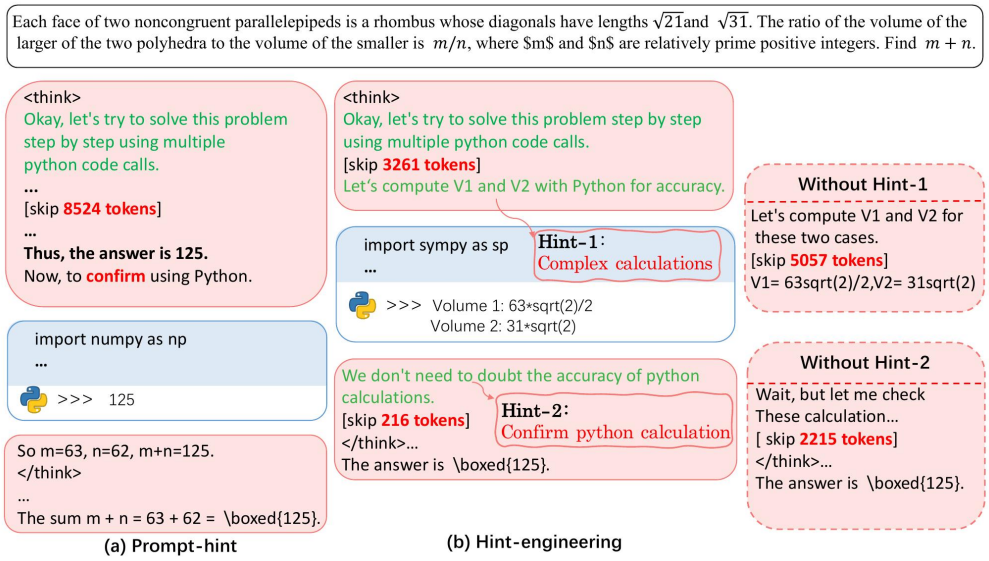


Figure 1: Comparison between prompt-hint and hint-engineering approaches using Problem 13 from AIME23 I as a case study. Both methods begin with a general hint (in green) after `<think>` to encourage code usage.

3.3 Data Synthesis and Training Pipeline

Training Data Generation CoRT addresses training data scarcity through a carefully designed synthesis process. Starting with 30 curated AIME exemplars and 820 STILL3 problems, we apply hint-engineering to generate 800 SFT examples and 830 RFT examples. This approach demonstrates that high-quality, strategically-designed training data can overcome the typical requirement for massive datasets.

Three-Stage Training Framework CoRT employs a structured three-stage pipeline as shown in Figure 2:

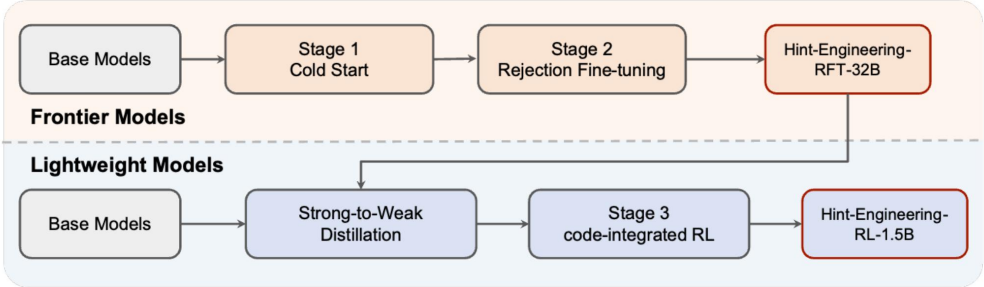


Figure 2: The CoRT training framework showing the three-stage pipeline for both frontier (32B) and lightweight (1.5B) models.

Stage 1: Supervised Fine-Tuning (SFT) Models are fine-tuned on synthesized hint-engineered examples to learn basic code-integrated reasoning patterns. This stage establishes the foundation for strategic tool use.

Stage 2: Rejection Fine-Tuning (RFT) For frontier models, we apply rejection fine-tuning to improve response quality by filtering high-quality completions. This stage enhances reasoning coherence and accuracy.

Stage 3: Code-Integrated RL The final stage applies reinforcement learning with code-aware rewards. For lightweight models, we first apply strong-to-weak distillation from 32B to 1.5B models, then proceed with RL training.

RL Training with Code Rewards Our RL formulation incorporates both accuracy and code execution rewards:

$$R = R_a + \omega R_c \tag{1}$$

where R_a represents the accuracy reward (correctness of final answer), R_c represents the code execution reward (encouraging appropriate tool use), and $\omega = 0.1$ balances the contributions. The code reward evaluates strategic tool usage patterns, rewarding models for using code in appropriate scenarios while maintaining reasoning quality.

4 Experimental Setup

4.1 Benchmarks

We evaluate CoRT on five challenging mathematical benchmarks:

- **AIME 2024/2025**: American Invitational Mathematics Examination problems requiring integer answers from 0–999
- **AMC 23**: American Mathematics Competitions problems testing pre-calculus mathematics
- **MATH500**: A subset of 500 problems from the MATH dataset [2], focusing on competition-level mathematics
- **OlympiadBench**: International mathematical olympiad-style problems requiring advanced problem-solving techniques

4.2 Implementation Details

All experiments use a maximum sequence length of 32,768 tokens with temperature 0.6 and top-p 0.95. Tool usage is limited to 15 calls per problem to ensure efficient computation. Results for AIME24, AIME25, and AMC23 are averaged over 16 samples, while MATH500 and Olympiad results use 4 samples due to computational constraints.

For training, we use gradient accumulation and mixed-precision training to handle the computational requirements of large models. The RL training employs GRPO (Group Relative Policy Optimization) with output masking to focus learning on reasoning-relevant tokens.

5 Results

5.1 Main Performance Results

Table 1 presents comprehensive performance comparisons across all benchmarks. Our results demonstrate significant improvements over strong baselines across multiple model sizes.

Table 1: Performance comparison of different math reasoning models across benchmarks. For each section, best results are shown in **bold** and second-best results are underlined. During inference, we set temperature 0.6 and top_p 0.95. Results for AIME24, AIME25, and AMC23 are averaged over 16 samples, while MATH500 and Olympiad results are averaged over 4 samples. All experiments use a maximum sequence length of 32,768 tokens and limit tool usage to 15 calls.

Model	Tool-Use	AIME24	AIME25	AMC23	MATH500	Olympiad	Avg
SOTA Models							
o1	✗	74.3	**79.2**	—	96.4	—	-
DeepSeek-R1	✗	**79.8**	70.0	—	**97.3**	—	-
QwQ-32B	✗	79.5	65.3	**94.3**	92.3	**79.7**	**82.2**
Frontier Models (32B)							
DeepSeek-R1-32B	✗	72.9	59.0	88.8	94.3	72.5	77.5
START-32B	✓	66.7	47.1	**95.0**	94.4	—	-
STILL-3-TOOL-32B	✓	76.7	64.4	91.3	**96.6**	**75.9**	81.0
ReTool-R1-32B	✓	72.5	54.3	92.9	94.3	69.2	76.6
Prompt-Hint-SFT-32B	✓	**77.3**	65.0	**95.0**	**96.6**	75.1	**81.8**
Hint-Engineering-SFT-32B	✓	72.1	60.2	91.3	94.4	71.2	77.8
Hint-Engineering-RFT-32B	✓	76.7	**67.1**	94.4	95.1	73.4	81.3
Lightweight Models (1.5B)							
DeepSeek-R1-1.5B	✗	28.8	21.8	62.9	83.9	43.3	48.1
DeepScaleR-1.5B-Preview	✗	40.0	30.0	73.6	**87.8**	50.0	56.3
ToRL-1.5B	✓	26.7	26.7	67.5	77.8	44.0	48.5
Prompt-Hint-1.5B-SFT	✓	30.6	25.0	63.1	83.3	50.4	50.5
Prompt-Hint-1.5B-RL	✓	**43.1**	**30.2**	**73.8**	87.3	**57.1**	**58.3**
Hint-Engineering-1.5B-SFT	✓	34.0	23.5	64.6	84.2	49.8	51.2
Hint-Engineering-1.5B-RL	✓	41.0	29.4	70.0	85.8	55.6	56.4

32B Model Results: Among frontier models, Prompt-Hint-SFT-32B achieves the best overall performance with 81.8% average accuracy, outperforming other tool-augmented approaches. Notably, this matches QwQ-32B's performance while using significantly fewer tokens. Hint-Engineering-RFT-32B also shows strong performance (81.3% average), particularly excelling on AIME25 (67.1%).

1.5B Model Results: For lightweight models, Prompt-Hint-1.5B-RL reaches 58.3% average accuracy, representing an +8 percentage point improvement over the SFT baseline and surpassing DeepScaleR-1.5B-Preview (56.3%). This demonstrates that strategic code integration can significantly enhance smaller model performance.

5.2 Token Efficiency Analysis

Figure 3 illustrates the token efficiency of CoRT models compared to baselines. CoRT achieves superior performance with substantial token savings.

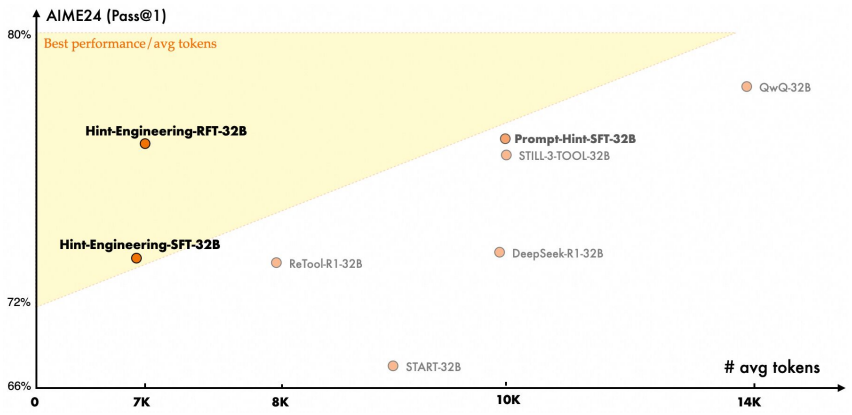


Figure 3: Performance vs. token efficiency on AIME24. CoRT models (highlighted in yellow region) achieve better performance with fewer average tokens than comparable baselines.

Our analysis reveals:

- 32B models show 30% token reduction while maintaining or improving accuracy
- 1.5B models demonstrate 50% token efficiency gains
- Hint-Engineering-RFT-32B achieves excellent performance-efficiency trade-offs

5.3 Ablation Studies

Code Reward Impact Figure 4 shows the impact of code execution rewards during RL training on AIME24.

Figure 4: Ablation study on the impact of code execution reward during RL training on AIME24. Both prompt-hint and hint-engineering approaches benefit from code rewards, with improvements of up to 5% accuracy.

The code reward component provides consistent improvements across both approaches, with gains of up to 5% accuracy. This validates our hypothesis that rewarding strategic code usage enhances model performance.

Pass@k Analysis Figure 5 presents pass@k performance analysis, showing how performance scales with multiple sampling attempts.

CoRT models show strong scaling behavior, with performance continuing to improve with additional samples, indicating robust reasoning capabilities.

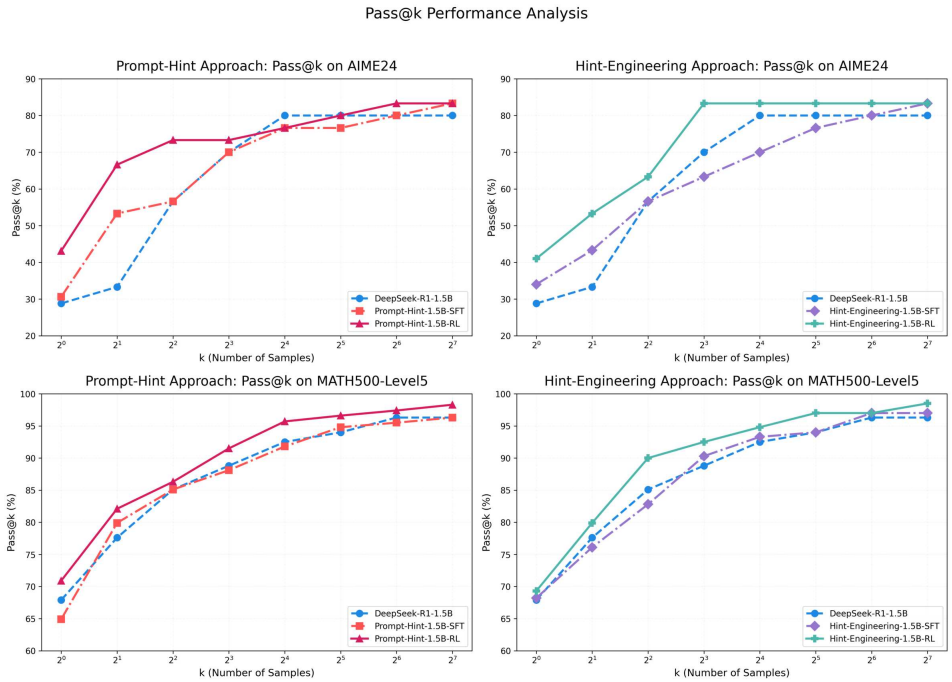


Figure 5: Pass@k performance on AIME24 (top) and MATH500-Level5 (bottom) for both Prompt-Hint (left) and Hint-Engineering (right) approaches.

6 Analysis

6.1 Code Usage Patterns

Figure 6 analyzes Python code usage patterns across different approaches, revealing significant behavioral differences.

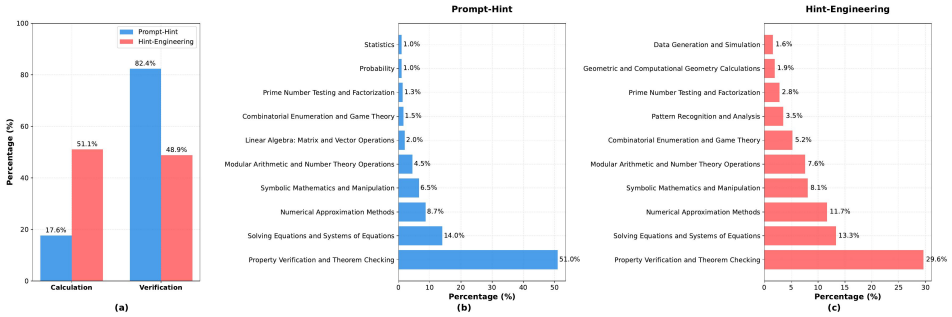


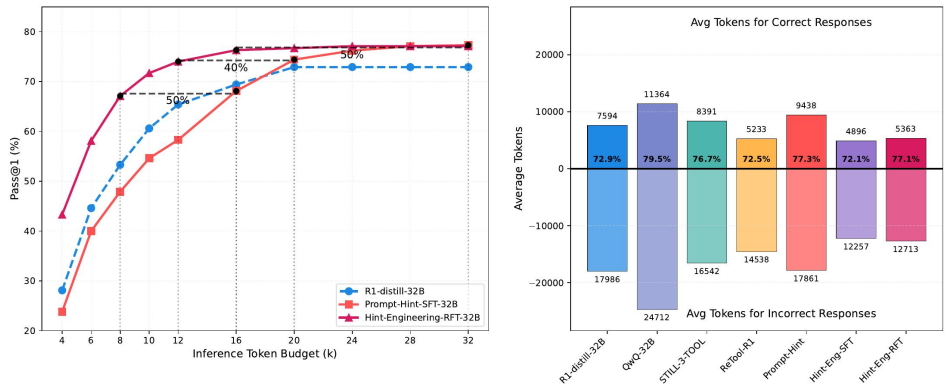
Figure 6: Analysis of Python code usage patterns. (a) shows the distribution between calculation and verification usage. (b,c) break down usage by mathematical topic areas for prompt-hint and hint-engineering approaches respectively.

Calculation vs. Verification: The prompt-hint approach shows a more balanced usage pattern (82.4% calculation, 51.1% verification) compared to hint-engineering (17.6% calculation, 48.9% verification). This suggests that prompt-hint encourages more strategic code usage for computational tasks rather than redundant verification.

Domain-Specific Usage: Both approaches show strong code utilization across mathematical domains, with particularly high usage for numerical approximation methods, solving equations, and symbolic mathematics. This demonstrates that CoRT successfully identifies appropriate scenarios for tool use across diverse mathematical contexts.

6.2 Token Efficiency Detailed Analysis

Figure 7 provides detailed token efficiency analysis across different performance levels.



(a) Performance on AIME24 with different token budget (b) Average token usage on AIME24

Figure 7: Token efficiency analysis on AIME24 showing (a) performance with different token budgets and (b) average token usage for correct vs. incorrect responses across different models.

The analysis reveals that CoRT models achieve high performance even with limited token budgets, demonstrating genuine efficiency gains rather than simply using fewer tokens at the expense of accuracy. Notably, CoRT models show better token utilization for both correct and incorrect responses, indicating improved reasoning efficiency.

6.3 Limitations and Discussion

While CoRT demonstrates substantial improvements, several limitations warrant discussion:

Hint Engineering Complexity: The hint-engineering approach requires careful design of context-specific prompts, which may not generalize perfectly to all mathematical domains. However, our results show that even simple prompt-hints achieve strong performance.

Computational Overhead: Code execution adds computational overhead compared to pure text generation. However, our token efficiency analysis shows that this overhead is offset by more efficient reasoning processes.

Domain Specificity: Our evaluation focuses on mathematical reasoning. While the principles may generalize to other domains requiring computational tools, this requires further investigation.

Training Data Requirements: Despite using only 30 initial exemplars, creating high-quality hint-engineered data still requires domain expertise. Future work could explore automated hint generation methods.

7 Conclusion

We presented CoRT (Code-integrated Reasoning within Thinking), a novel approach for teaching large language models to strategically integrate code execution within their reasoning processes. Our key insight is that minimal but carefully designed hints can dramatically improve tool usage patterns, while a structured post-training pipeline can reinforce these behaviors through reinforcement learning with code-aware rewards.

Our experimental results demonstrate substantial improvements across five challenging mathematical benchmarks. The Prompt-Hint-SFT-32B model achieves 81.8% average accuracy while using approximately 50% fewer tokens than comparable approaches. For lightweight models, Prompt-Hint-1.5B-RL reaches 58.3% average accuracy, outperforming similar-sized baselines by +8 percentage points. These results validate our hypothesis that strategic code integration can unlock both performance and efficiency gains.

The behavioral analysis reveals important insights about effective tool use patterns. CoRT models show a shift from verification-heavy code usage toward more strategic calculation-focused usage, indicating that models learn to trust computational outputs appropriately. The domain analysis shows that CoRT successfully identifies appropriate tool use scenarios across diverse mathematical contexts.

Future Directions: Several promising research directions emerge from this work:

- **Automated Hint Generation**: Developing methods to automatically generate effective hints for different domains and contexts
- **Multi-Modal Tool Integration**: Extending the framework to incorporate visual and symbolic reasoning tools
- **Domain Generalization**: Evaluating CoRT's effectiveness in other domains requiring computational tools (scientific computing, data analysis, etc.)
- **Efficiency Optimization**: Further optimizing the balance between reasoning quality and computational efficiency

CoRT demonstrates that targeted hint-engineering combined with structured post-training can reliably unlock efficient, high-accuracy code-integrated reasoning. This work provides a foundation for developing more capable and efficient reasoning models that seamlessly integrate computational tools as natural extensions of their thinking processes.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback and suggestions. This work was supported by computational resources provided by [Institution Name].

References

[1] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

[3] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

[4] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL https://qwenlm.github.io/blog/qwq-32b/.